

From Messy Links to One Clean MP4 File

Yeh guide is class ka "theory wala" hissa hai -- notebook abhi nahi khulega. Isme samjhaya gaya hai ki yeh project hai kya, kaunsi library aur pattern iske peeche kaam kar rahe hain, aur agli class mein jab asli code saamne aayega, tab har line pehchani hui lagni chahiye.

THE ECOSYSTEM

REGEX FROM ZERO

THE FUNCTIONS

THE EVENT FLOW

LIMITATIONS

SETUP FOR CLASS 2

IS GUIDE KE ANDAR

Before Before We Begin

Yeh class kya pehle se jaanti hai maan kar chal rahi hai, aur kaunse naye words aayenge.

Part 1 Why This Project Exists

Asli problem kya hai -- alag-alag sites, khatarnak download buttons, aur bharosemand file na milna.

Part 2 The Ecosystem

Har library kaun si hai, kya built-in hai kya installed, aur poora flow paanch steps mein.

Part 3 A RegEx Primer

RegEx bilkul zero se, phir woh teen exact patterns jo yeh project use karta hai.

Part 4 The Event Layer

Ek poora run, shuru se aakhir tak, ek kahani ki tarah -- Meher ke saath.

Part 5 The Technical Layer

Har function -- kya leta hai, kya deta hai, aur peeche ki soch kya hai.

Part 6 What This Cannot Do

Limitations, jahan yeh fail ho sakta hai, aur live demo se pehle kya check karna hai.

Part 7 Before Class 2

Sab kuch install karna, aur isi idea ko ek plain .py script ki tarah chalana.

BEFORE WE BEGIN

What This Class Assumes

Ek chhoti si shuruaati page, taaki kuch bhi technical shuru hone se pehle sabki expectations ek jaisi ho jaayein.

Yeh class maan kar chal rahi hai ki core Python aapke liye naya nahi hai -- variables, data types, `if` / `else`, loops, dictionaries, aur ek function likhna jo arguments le aur kuch return kare. Yahan yeh saari cheezein dobara nahi sikhayi jaayengi, aur na hi yeh guide unki jagah leta hai.

Is guide mein jo naya hai, woh us buniyaad ke *upar* baitha hai: ek external library jo video platforms se "baat karna" jaanti hai, code likhne ki ek chhoti si aadat jisse woh galat input par kabhi crash na ho, aur ek underused built-in tool -- **regular expressions** -- jo yahan messy text ke andar links dhoondhne aur filenames ko safe banane ke liye use hota hai.

Code poore guide mein baar-baar aayega, lekin hamesha uske concept ke baad, kabhi pehle nahi -- aur pehli baar mein ise yaad karne ki koi zaroorat nahi. Agar kabhi explanation aur uske saath wala code disconnected lage, to samajh lijiye woh gap in pages ka hai, aapka nahi -- use note kar lijiye aur class mein pooch lijiye.

Aap jaante hain: Python basics



Yeh guide sikhayega: library + RegEx + safe habits



Agli class: asli notebook code

WOH WORDS JO IS GUIDE MEIN MILENGE

Ek baar sirf sarsari nazar se padh lijiye -- seekhne ki koshish mat kijiye abhi. Har term aage poori tarah se samjhayi jaayegi -- yeh table bas isliye hai taaki koi bhi word bilkul anjaana na lage.

TERM	MOTA-MOTA MATLAB	KAHAN POORA SAMJHAYA JAAYEGA
Library	Ek ready-made, tested toolkit -- kisi aur ne likh kar publish kiya hua code	Part 2
RegEx	Text ke andar se kuch dhoondhne/match karne ke liye ek chhoti si pattern-language	Part 3
Widget	Notebook ke andar ek clickable UI cheez -- button, text box, upload area	Part 4
Metadata	Video ke baare mein jaankari (title, length) -- bina download kiye hi mil jaati hai	Part 4
Container / format	File ka "wrapper", jaise MP4, jo video aur audio dono ko andar rakhta hai	Part 5
Exception	Python ka tareeka yeh batane ka ki beech mein kuch galat ho gaya	Part 5

TERM	MOTA-MOTA MATLAB	KAHAN POORA SAMJHAYA JAAYEGA
Persistence	Ek value ko disk par save karna, taaki program band hone ke baad bhi woh yaad rahe	Part 5 & 7

EK WADA, DONO TARAF SE

Yeh guide kabhi kisi regex character, kisi design choice, ya kisi failure case ko chhota samajh kar skip nahi karega. Badle mein, kuch bhi bina samjhaye use nahi hoga. Agar padhne ke baad bhi kuch clear na ho, to woh in pages ki kami hai, aapki nahi -- likh lijiye aur class mein le aaiye.

PART 1

Why This Project Exists

Kisi bhi function ya library se pehle -- asli problem kya hai, aur yeh tool use kyun solve karta hai.

Chaliye ek simple si situation se shuru karte hain. Maan lijiye aapke paas teen links hain -- ek YouTube video, ek Instagram reel, aur ek Facebook video, teeno alag-alag dost ne bheje the. Ab in teeno ko apne phone ya laptop mein save karna hai, taaki internet na ho tab bhi dekhe ja sakein. Yahin se asli museebat shuru hoti hai -- kyunki teeno platforms ke liye, alag-alag download website use karni padti hai. Jo site YouTube ke liye kaam karti hai, woh Instagram ke link par ek dum kaam nahi karti, aur Facebook ke liye phir se ek naya site dhoondhna padta hai.

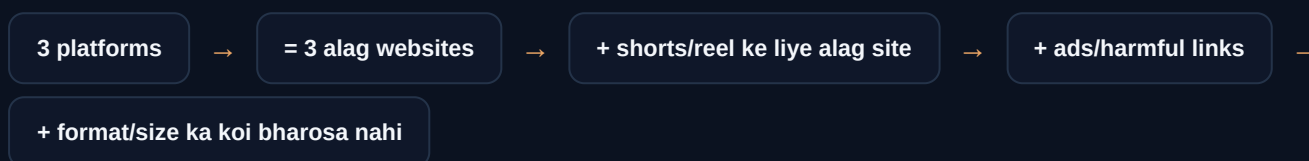
Aur museebat yahin khatam nahi hoti. Kabhi-kabhi to ek hi platform ke andar bhi do alag sites chahiye hoti hain -- ek poori lambi YouTube video download karne wali site alag hoti hai, aur wahi YouTube ka ek chhota Short ya Reel download karne ke liye kabhi-kabhi doosri site istemaal karni padti hai, kyunki pehli site Shorts ka link samajh hi nahi paati. Ek chhote se kaam ke liye -- teen-chaar videos download karne ke liye -- ek student ke paas paanch-chhah alag-alag download websites ke tabs khule reh jaate hain.

ASLI KHATRA: YEH "FREE" DOWNLOAD SITES

Ab is museebat ka doosra, zyada khatarnak hissa dekhte hain. Yeh open, free download websites zyadatar paisa "ads" se kamati hain -- aur bahut baar in ads ko is tarah design kiya jaata hai ki asli "Download" button dhoondhna hi mushkil ho jaaye. Screen par teen-chaar bade, chamakte hue "Download Now" button dikhte hain, lekin sirf ek asli hota hai -- baaki sab ads hain jo kisi aur (kabhi-kabhi harmful) site par le jaate hain. Kayi baar to button dabane ke baad ek naya tab khul jaata hai jo kuch aur hi promise karta hai -- ek prize, ek app install, ya ek "aap winner hain" jaisa message -- jo seedha kisi harmful ya scam website tak le jaata hai. Ek dusri video download karne ki koshish mein, poora system risk mein daal dena -- yeh koi acchi deal nahi hai.

AUR PHIR, FILE BHI BHAROSEMAND NAHI HOTI

Maan lijiye sahi button mil bhi gaya aur download ho bhi gaya -- ab teesri problem shuru: file hamesha `.mp4` mein hi nahi aati. Kabhi woh `.webm` mein aati hai jo har phone/app mein khulti nahi. Quality ka bhi koi bharosa nahi -- kabhi video itna compress ho jaata hai ki dekhna mushkil ho jaata hai. Aur size bhi kaafi reckless tareeke se handle hota hai -- ek do-minute ki reel kabhi 3 MB aati hai, aur kabhi wahi jaisi reel 80 MB ki bhaari file ban kar aati hai, bina kisi control ke.



YEH SCRIPT ISI POORI KAHANI KO SOLVE KARTA HAI

Ab socho -- agar yeh saara kaam ek hi jagah, bina kisi ad, bina kisi extra website ke ho jaaye to? Yehi is Python project ka idea hai. Ek hi script, teeno platforms (YouTube, Instagram, Facebook) ke links samajhti hai -- chahe woh poori video ho ya chhota Short/Reel, script khud pehchaan leti hai ki kaunsa hai aur uske hisaab se sahi tareeke se download karti hai. Koi third-party website nahi, koi ad, koi "Download Now" button jo kahin aur le jaaye -- bas ek link, aur seedha ek clean file.

1 Ek jagah, teen platforms

Ek hi script YouTube, Instagram, aur Facebook, teeno ke links samajh sakti hai -- alag-alag websites dhoondhne ki zaroorat khatam.

2 Poori video ho ya Short/Reel, farak nahi padta

Script khud pehchaan leti hai ki link kis type ka hai, aur usi hisaab se sahi tareeke se kaam karti hai.

3 Koi ad, koi harmful redirect nahi

Yeh ek Python code hai jo directly platform se baat karta hai -- beech mein koi ad-bhara website nahi aati, isliye koi bhatakne wala button bhi nahi.

4 Hamesha ek clean .mp4 file

Chahe platform ne video ko andar kaise bhi bheja ho, script use hamesha ek single, ready-to-play MP4 file mein badal deti hai.

5 Size ka dhyaan rakha jaata hai

File ka size samajh-boojh kar handle kiya jaata hai, bina kisi random 5x badhi hui file ke.

TOH FAYDA KYA HAI, SEEDHE SHABDON MEIN?

Jo insaan pehle paanch alag websites khol raha tha, ad-buttons se bach raha tha, aur phir bhi kabhi kabhi galat format ki file paa raha tha -- ab woh sirf ek jagah apne saare links daal deta hai, ek button dabata hai, aur thodi der baad uske paas har video ki ek saaf, ready MP4 file hoti hai. Kaam wahi hai jo pehle 20 minute leta tha, ab ek automated batch job hai jo khud chal kar khatam ho jaata hai.

ISKE PEECHE KA PYTHON IDEA

Yeh project dikhata hai ki Python ka asli fayda kya hai -- alag-alag chhote, bharosemand tools (libraries) ko jodkar ek badi, useful cheez banana. Video download karne ka bhaari kaam ek trusted library (`yt_dlp`) karti hai; Python ka kaam hai us library ko sambhalna -- messy input samajhna, galat cheezein pakadna, aur sab kuch ek simple interface ke peeche chhupa dena. Yeh soch aage Part 2 mein पूरी तरह khulegi.

PART 2

The Ecosystem This Project Stands On

Python mein kya built-in hai, kya udhaar liya gaya hai, aur har piece apni jagah kyun deserve karta hai.

Har ecosystem ek language ke around bana ek workshop hota hai. Class mein ab tak sikhaya gaya core Python, khud ek workbench hai -- solid, general-purpose, aur agar time diya jaaye to kuch bhi bana sakta hai. Ek library us workbench mein ek specialised, pehle se bana hua power-tool add karti hai -- kisi aur ne likha aur test kiya hua, jise hum use karte hain, dobara banate nahi.

LIBRARY	IS PROJECT MEIN ROLE	KAHAN SE AATI HAI
<code>yt_dlp</code>	Asli extraction/download engine -- YouTube, Instagram, aur Facebook, teeno ke liye	Installed (pip)
<code>ipywidgets</code>	Upload button, text box, progress bar, aur results table banati hai	Installed (pip)
<code>os</code>	Folder banana, path handle karna, file exist karti hai ya nahi check karna	Built-in
<code>re</code>	Messy text ke andar links dhoondhna, aur filenames ko saaf karna	Built-in
<code>urllib.parse</code>	Poore URL mein se sirf domain name nikaalna	Built-in
<code>datetime</code>	Summary report file ko timestamp dena	Built-in
<code>json</code>	Chuna hua destination folder, do runs ke beech save/re-read karna	Built-in
<code>IPython.display</code>	Notebook cell kya dikhaye -- tables, banners, refresh -- yeh control karta hai	Jupyter ke saath aata hai

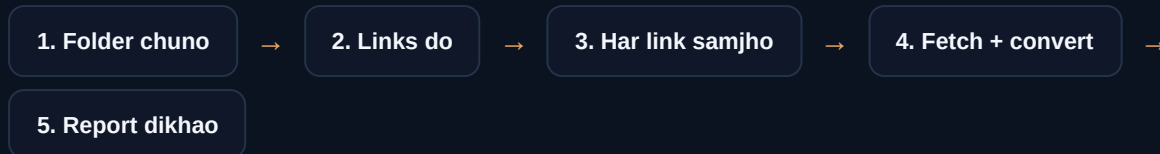
DO IMPORTS JO KABHI USE HI NAHI HOTE

Project ke pehle version mein `io` aur `traceback` bhi import kiye gaye hain -- lekin dono kahin bhi actually use nahi hote. `traceback` ko aage properly wire karna faayde ka rahega: yeh error ka poora technical detail print kar sakta hai, sirf ek-line summary nahi -- jo kaam aata hai jab yeh project classroom demo se aage badh kar real debugging tak pahunchta hai. Ek chhota, honest example hai ki working code mein bhi kaise leftover lines reh jaati hain -- aur ek reminder ki har import ko padho, sirf bharosa mat karo.

PART 2 · CONTINUED

The Five-Step Flow Every Run Follows

Kisi bhi function ko dekhne se pehle, poore pipeline ko ek saath dekh lena helpful hota hai. Is project ka har run -- chahe usme kitne bhi links kyun na ho -- inhi paanch steps se, isi order mein guzarta hai.



STEP	KYA HOTA HAI	KAUN SAMBHALTA HAI
1. Destination dena	User ek folder chunta hai ya confirm karta hai jahan files save hongi	<code>ipywidgets</code> , <code>os</code> , <code>json</code>
2. Links dena	User ek .txt file upload karta hai -- kaisa bhi naam ho, kaisi bhi structure ho andar	<code>ipywidgets</code>
3. Har link samajhna	Platform aur video type pehchana jaata hai; us platform se metadata maanga jaata hai	<code>re</code> , <code>urllib.parse</code> , <code>yt_dlp</code>
4. Fetch aur convert	Video download hota hai aur ek single .mp4 file mein merge kiya jaata hai	<code>yt_dlp</code> (peeche <code>ffmpeg</code> ke saath)
5. Report	Live results table update hota hai, aakhir mein ek summary file save hoti hai	<code>ipywidgets</code> , <code>IPython.display</code> , <code>datetime</code>

Yeh dhyaan dijiye ki har step ab bhi wahi core Python cheezon par tika hai jo aap pehle se jaante hain -- functions, dictionaries, loops, file handling -- bas kuch udhaar liye tools ke around carefully arrange kiya gaya hai. Poore pipeline mein kahin bhi aisi cheez nahi hai jo is class ki jaankari se bahar ho, sivaay us chhote se naye hisse ke jo yeh guide sikha raha hai.

PAANCH STEPS KYUN, EK BADA FUNCTION KYUN NAHI

Pipeline ko paanch alag responsibilities mein todna khud ek design lesson hai. Har step ko akela test kiya ja sakta hai, samjhaya ja sakta hai, aur fix kiya ja sakta hai, doosron ko chhue bina -- Part 5 mein exactly dekhenge ki kaunsa function kaunsa step sambhalta hai, aur unhe alag rakhna kyun zaroori hai.

PART 3

Reading Regular Expressions, From Zero

Ek pattern-matching language jo is project mein do jagah use hoti hai -- pehle ise achhe se samajh lete hain. Regular expression -- chhote mein regex -- ek chhoti si pattern-language hai jo text ki *shape* describe karti hai, taaki Python us shape ko dhoondh sake, ek exact fixed copy ki jagah. Normal check poochta hai "kya yeh string bilkul isi ke barabar hai?" -- regex poochta hai "kya yeh string is pattern jaisi dikhti hai?" -- aur yehi chahiye jab is project mein aane wala text unpredictable aur messy ho.

BUILDING BLOCKS

SYMBOL	MATLAB	EXAMPLE MATCH
Plain letters	Bilkul apne aap ko hi match karte hain	<code>cat</code> match karega "cat"
<code>.</code>	Koi bhi ek character	<code>c.t</code> match karega "cat", "cut", "c9t"
<code>\s</code>	Koi bhi whitespace -- space, tab, newline	do words ke beech ka gap
<code>\d</code>	Koi bhi ek digit, 0-9	<code>\d\d</code> match karega "42"
<code>[...]</code>	Is set mein se koi bhi ek character	<code>[aeiou]</code> ek vowel match karega
<code>[^...]</code>	Is set mein <i>na</i> hone wala koi bhi ek character	<code>[^\s]</code> whitespace ke alawa kuch bhi
<code>*</code>	Pehle wali cheez, zero ya usse zyada baar	<code>ab*</code> match karega "a", "ab", "abbb"
<code>+</code>	Pehle wali cheez, ek ya usse zyada baar	<code>ab+</code> match karega "ab", akela "a" nahi
<code>?</code>	Pehle wali cheez, zero ya ek baar -- optional bana deta hai	<code>colou?r</code> match karega "color" aur "colour" dono


```
import re

re.findall(r"\d+", "Room 42, Table 7")
# Result: ['42', '7']
# \d+ = "ek ya zyada digits, aapas mein chipke hue"
# re.findall() dhoondhta hai HAR jagah jahan pattern match ho raha hai,
# aur sabko ek Python list mein wapas de deta hai.

re.findall(r"[A-Z]\w+", "Raaj Deep Ray")
# Result: ['Raaj', 'Deep', 'Ray']
# [A-Z] = ek capital letter se shuru hona chahiye
# \w+ = uske baad ek ya zyada "word characters" (letters/digits/underscore)

re.findall(r"\d{2}:\d{2}", "Class 9:30 se 11:15 tak hai")
# Result: ['9:30', '11:15']
# \d{2} = exactly do digits (yahan {2} likhna '*' ya '+' se zyada precise hai)
```

Ek habit turant apna lijiye: Python mein regex patterns lagbhag hamesha `r"..."` ki tarah likhe jaate hain -- ek **raw string**. Agar `r` na likhein, to Python `\s` ko regex tak pahunchne se pehle hi apna khud ka escape sequence samajh kar interpret karne ki koshish karega. Raw-string prefix Python ko bolta hai: "har backslash ko bilkul jaisa likha hai waisa hi rehne do."

PART 3 · CONTINUED

The Exact Patterns This Project Uses

Building blocks samajhne ke baad, ab dekhte hain woh exact teen patterns jin par yeh project tika hai -- piece by piece todkar, phir simple English/Hinglish mein dobara.

PATTERN 1 · MESSY TEXT KE ANDAR SE LINKS DHOONDHNA

```
r"https?://[^\s]+"
```

PIECE	MATLAB
<code>https</code>	Literal characters -- bilkul aise hi likhe hone chahiye
<code>s?</code>	Letter s, optional -- isliye http aur https, dono match hote hain
<code>://</code>	Phir se literal characters
<code>[^\s]+</code>	Ek ya zyada characters jo whitespace NAHI hain -- baaki poora link, kitna bhi lamba ho

Seedhe shabdon mein: *kuch bhi dhoondho jo http:// ya https:// se shuru ho aur agli space tak chalta rahe*. Yehi hai jo `parse_links()` ko ek file ke andar se URLs nikaalne deta hai -- chahe numbering ho, bullets ho, ya aas-paas prose likha ho.

PATTERN 2 · WOH CHARACTERS HATANA JO FILENAME TODH DENGE

```
r'[\\/*?:"<>|]'
```

Yeh ek character class hai jo ek-ek karke har woh character list karta hai jo Windows mein filename ke andar allow nahi hai -- backslash, forward slash, asterisk, question mark, colon, quotation mark, aur angle brackets/pipe.

`re.sub()` ke saath jodkar, is set ka har character ek khaali string se replace ho jaata hai -- yaani video ke title se delete ho jaata hai, filename banne se pehle.

PATTERN 3 · EXTRA SPACES KO EK SPACE MEIN SAMET DENA

```
r"\s+"
```

Ek ya zyada whitespace characters, ek single unit ki tarah match karke, exactly ek space se replace kar diye jaate hain -- toh "My Trip to Jaipur" jaisa irregular-spaced title ban jaata hai `"My Trip to Jaipur"`.

REGEX YAHAN KYUN, .SPLIT() YA .REPLACE() KYUN NAHI?

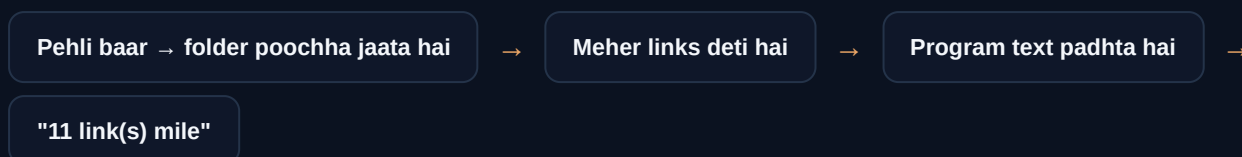
Plain string methods tab kaam karte hain jab dhoondhi jaane wali exact value pehle se maloom ho. Woh asaani se *anjaan lambai* ka anjaan text express nahi kar paate ("http://" ke baad jo bhi aaye, kitna bhi lamba ho"), ya "is poore forbidden set mein se koi bhi ek character." Regex ek fixed value ki jagah ek shape express karta hai -- aur yehi chahiye hota hai asli, messy, insaano ke type kiye hue text ke liye.

PART 4

What Happens, In The Order It Happens

Ek realistic run ki kahani, bina ek bhi line code ke.

Ecosystem aur regex ki buniyaad tayyar hone ke baad, ab is section mein ek poore run ki kahani sunte hain -- jis order mein cheezein actually hoti hain, abhi tak koi code nahi dikhaya jaayega. Isse concrete banane ke liye, socho Meher ke baare mein -- second-year ki ek student, jiske paas WhatsApp chat se save kiye hue barah links hain: kuch YouTube Shorts, ek Instagram reel, aur ek Facebook video jo uske classmates ne share ki thi.



SABSE PEHLE · FOLDER WALA SAWAAL

Jab notebook kisi machine par pehli baar chalti hai, uske paas abhi files rakhne ki koi saved jagah nahi hoti. Upload option aane se pehle, ek text box ek destination folder poochhta hai, saath mein ek simple yes-ya-no choice: *isse next time ke liye default bana dein?* Agar Meher "yes" chunti hai, to woh path notebook ke saath rakhi ek chhoti si settings file mein likh diya jaata hai. Agli baar jab woh notebook kholti hai -- chahe hafton baad -- wahi folder already bhara hua milta hai, confirm ya override karne ke liye taiyaar.

MEHER APNE LINKS DETI HAI

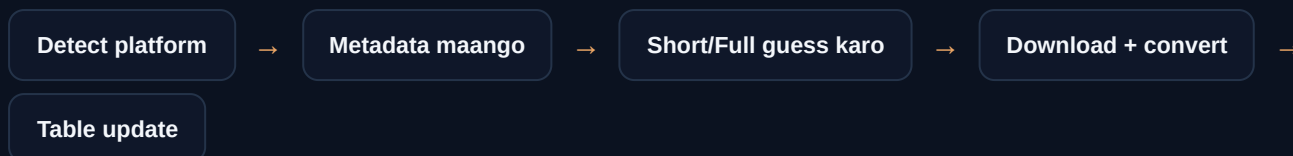
Meher apne barah links Notepad mein paste karti hai, saath mein do-teen line bhi -- jaise "yeh dekho guys" -- aur file ko `clips.txt` naam se save kar deti hai, ek naam jo usne bina zyada socha chuna. Upload widget ko farak nahi padta ki usne kya naam diya -- sirf `.txt` extension maayne rakhta hai. Woh Upload par click karti hai, file chunti hai, aur aadhe second intezaar karti hai.

PROGRAM LINE KE BEECH PADHTA HAI

Us aadhe second ke peeche, uploaded file ka raw text decode hota hai aur Part 3 wale link-dhoondhne wale regex pattern ko diya jaata hai. Woh Meher ka woh stray sentence bilkul ignore kar deta hai, exactly barah addresses nikaal leta hai, ek accidental duplicate hata deta hai jo usne galti se do baar paste kiya tha, aur wapas batata hai: ek green confirmation banner mein "11 link(s) found". Ab hi Start button clickable banta hai -- program kisi ko bhi Start dabane nahi deta jab tak kaam karne layak kuch valid na ho.

PART 4 · CONTINUED

One Link At A Time, To A Finished Report



EK WAQT MEIN EK LINK

Meher Start dabaati hai. Program gyaarah links ko ek saath nahi chhu deta -- woh unhe ek-ek karke process karta hai, aur har link ke liye, wahi chhoti si sequence dohraata hai: domain padh kar decide karo yeh kaunsa platform hai; us platform se video ke baare mein basic jaankari maango, bina kuch download kiye abhi; guess karo ki yeh short-form clip hai ya poori video; phir usse actually fetch karo aur convert karke ek single MP4 file banao, jiska naam video ke apne title se banaya gaya ho, usmein se unsafe cheezein hata kar.

HOTE HUE DEKHNA

Har link process hone ke saath progress bar chadhata jaata hai, aur uske neeche results table har baar khud ko redraw karta hai -- green rows ek clean success ke liye, amber ek aisi file ke liye jo download to ho gayi lekin size limit se thodi upar chali gayi, aur red kisi bhi fail hone wali cheez ke liye, jiska asli reason seedhe uske paas plain text mein likha hota hai. Meher ke ek link par red aata hai: Instagram reel uske owner ne, link save karne ke baad kisi waqt delete kar diya tha. Run rukta nahi. Woh bas is failure ko note kar leta hai aur agle link par badh jaata hai.

AAKHRI BAAT

Jab aakhri link khatam hota hai, progress bar green ho jaata hai, aur ek chhota sa summary dikhta hai: kitne succeed hue, kitne fail hue, aur uske chuni hui folder mein save hui ek poori text report ka exact filename -- har video ka ek line, success ho ya failure, taaki poore run ke baare mein kuch bhi yaad se sochna na pade.

WOH PATTERN JO DHYAAN DENE LAYAK HAI

Meher ke barah mein se har link isi identical sequence se guzarta hai -- detect, probe, download, record -- chahe platform kuch bhi ho, outcome kuch bhi ho. Yeh sameness koi accident nahi hai; yehi hai jo sau links ka batch bhi utna hi manageable banata hai jitna das links ka. Part 5 batayega kaunsa function is sequence ke kaunsa step sambhalta hai.

PART 5 · 1 OF 6

Recognising A Link: Platform And Type

Neeche har function ek jaisi tarah se samjhaya gaya hai: kya leta hai, kya deta hai, aur usko aise hi kyun banaya gaya hai.

Leta hai: url -- ek string **Deta hai:** platform ka naam, string **Use karta hai:** urllib.parse

`detect_platform(url)` sirf URL ka domain dekh kar decide karta hai ki teen platforms mein se yeh kaunsa hai. Iske argument ki koi default value nahi hai -- URL hamesha dena hi padega.

```
def detect_platform(url: str) -> str:
    # Kaam: URL dekh kar bata do ki yeh YouTube, Instagram ya Facebook hai.
    # Input: url (string, zaroori hai) -- jaise "https://youtu.be/xyz123"
    # Output: platform ka naam -- "YouTube" / "Instagram" / "Facebook" / "Unknown"
    try:
        netloc = urlparse(url).netloc.lower()
        # urlparse() (urllib.parse se) URL ko tukdon mein todta hai.
        # .netloc matlab sirf domain wala hissa, jaise "www.youtube.com"
        # .lower() isliye taaki "YouTube.com" aur "youtube.com" dono match ho jaayein
    except Exception:
        return "Unknown"
        # URL hi kharab ho to crash mat karo -- seedha "Unknown" bhej do

    if "youtube.com" in netloc or "youtu.be" in netloc:
        return "YouTube"
    if "instagram.com" in netloc:
        return "Instagram"
    if "facebook.com" in netloc or "fb.watch" in netloc:
        return "Facebook"
    return "Unknown" # koi jaana-pehchana domain match nahi hua
```

Example: `detect_platform("https://youtu.be/xyz")` → "YouTube"

Ek aur example: `detect_platform("https://www.instagram.com/reel/abc123/")` → "Instagram", kyunki `netloc` mein `instagram.com` mil jaata hai.

Edge case jaanne layak: ek shortened link, jaise koi bit.ly address, bhi "Unknown" hi dega, kyunki asli destination domain tab tak chhupa rehta hai jab tak link ko actually follow na kiya jaaye.

Leta hai: url (string), info (yt-dlp se aaya dict) **Deta hai:** video-type label, string

Use karta hai: plain string checks + ek type guard

`detect_video_type(url, info)` guess karta hai ki link short-form clip hai ya poori video, pehle URL ke apne text se, aur zaroorat padhne par duration ke bharse.

```
def detect_video_type(url: str, info: dict) -> str:
    # Kaam: pata karo ki yeh Short/Reel hai ya ek Full video.
    # Input: url (string), info (dictionary -- yt-dlp se mila metadata, jaise duration)
    lowered = url.lower()

    if "/shorts/" in lowered:
        return "Short"
    if "/reel/" in lowered or "/reels/" in lowered:
        return "Reel/Short"
    # URL ke text se hi pata chal gaya -- info dictionary ki zaroorat nahi padi

    duration = info.get("duration") if info else None
    if isinstance(duration, (int, float)):
        # isinstance() check zaroori hai -- duration kabhi-kabhi None bhi ho sakta hai,
        # aur None <= 60 likhne se Python error de dega (TypeError)
        return "Short/Reel" if duration <= 60 else "Full video"

    return "Full video" # koi bhi signal na mile to default guess
```

Edge case jaanne layak: ek genuinely poori video jo sixty seconds se kam ki nikle -- maan lijiye ek movie trailer -- use bhi "Short/Reel" hi label kar diya jaayega. Yeh ek heuristic hai, koi 100% guarantee nahi.

PART 5 · 2 OF 6

Making A Title Safe To Save

Leta hai: name (string), max_len (int, default 120)

Deta hai: ek safe filename, string

Use karta hai: re.sub

`sanitize_filename(name, max_len=120)` is liye exist karta hai kyunki video ka title -- woh text jise yeh project apni saved file ka naam banane ke liye use karta hai -- internet se aata hai, aur use ek valid filename hone ka bharosa nahi diya ja sakta. `max_len` ka default 120 hona is liye hai kyunki zyadatar time default hi theek rehta hai; ise override sirf kisi unusual case mein karna padta hai -- aur yehi wajah hoti hai ki koi parameter required hone ki jagah ek default value paata hai.

```
def sanitize_filename(name: str, max_len: int = 120) -> str:
    # Kaam: title ko ek safe filename mein badlo -- har OS (Windows/Mac/Linux) par chale.
    # Input: name (string) -- video ka title; max_len (int) -- kitna lamba naam allowed hai
    if not name:
        name = "untitled_video"
        # Khaali ya None title aaya to seedha ek default naam de do

    name = re.sub(r'[/\*?:"<>|]', "", name)
    # Pattern 2 (Part 3B) -- Windows mein forbidden characters hata do.
    # re.sub(pattern, "", name) -- jahan pattern match ho, wahan "" (khaali) se badal do

    name = name.strip().strip(".")
    # Aage-peeche ke spaces/dots hata do (Windows end mein dot/space pasand nahi karta)

    name = re.sub(r"\s+", " ", name)
    # Pattern 3 (Part 3B) -- kayi spaces ek jagah aaye to unhe ek single space bana do

    if len(name) > max_len:
        name = name[:max_len].rstrip()
        # Bahut lamba title (Reels ke captions jaise) -- kaat do, warna file path
        # limit (khaaskar Windows par) todkar error de sakta hai

    return name or "untitled_video"
    # Sab kuch hata dene ke baad agar kuch bacha hi nahi, to default naam
```

Example: `sanitize_filename('My Trip????.mp4:2024')` → `'My Trip.mp4 2024'`

YEH CHHOTA SA FUNCTION DIKHTA CHHOTA HAI, PAR MATTER BAHUT KARTA HAI

Yeh woh ek jagah hai jahan kisi anjaan insaan ne internet par jo bhi video ka title type kiya tha, woh asal mein hard disk par likhi jaane wali cheez ban jaata hai. Ise skip karna, ya galat likhna, waisi hi chhoti si kami hai jo poore run ko crash kara sakti hai -- pehli hi baar jab kisi ke reel ke title mein ek colon ya slash aa jaaye, jo Instagram captions mein aksar hota hi hai, rare nahi.

PART 5 · 3 OF 6

Finding Every Link In A Messy File

Leta hai: `raw_text` -- poori file content, ek string **Deta hai:** URLs ki ek list, order same, duplicates hatakar

Use karta hai: `re.findall`, ek set, aur ek list saath mein

`parse_links(raw_text)` uploaded `.txt` file ka poora content ek hi string ki tarah leta hai, aur ek saaf URLs ki list return karta hai -- chahe numbering ho, bullet points ho, ya beech mein extra sentences mile hue ho.

```
def parse_links(raw_text: str) -> list:
    # Kaam: messy text ke andar se saare valid links nikaalo, duplicates hata ke.
    # Input: raw_text (string) -- poori uploaded file ek hi variable mein
    # Output: list of URLs (order same, koi bhi link do baar nahi)

    raw_matches = re.findall(r"https?://[^\s]+", raw_text)
    # Pattern 1 (Part 3B) -- text mein jahan bhi yeh shape mile, wahan se URL nikaal lo.
    # re.findall() poore text mein dhoondhta hai aur SAARE matches ek list mein deta hai --
    # ek line ho ya poora paragraph, koi farak nahi padta.

    cleaned = []
    seen = set()
    trailing_junk = ".,:;)]}'"

    for url in raw_matches:
        url = url.rstrip(trailing_junk)
        # copy-paste karte waqt link ke peeche comma/bracket chipak jaata hai -- hata do

        if url and url not in seen:
            seen.add(url)      # set() add fast hai -- "dekha hai?" turant pata chalta hai
            cleaned.append(url) # list isliye, kyunki set order yaad nahi rakhta hai

    return cleaned
```

EK SET AUR EK LIST, DONO SAATH MEIN KYUN

Ek `set` is sawaal ka jawab lagbhag turant deta hai: "kya yeh exact URL pehle dekha ja chuka hai?" -- chahe links kitne bhi ho. Lekin sets ko yaad nahi rehta ki items kis order mein add hue the -- aur file mein links jis order mein aaye the, woh order rakhna zaroori hai, taaki results table waisi hi sequence mein padha ja sake jaisa student ne paste kiya tha. Fast membership check ke liye ek set, aur order ke liye ek list -- yeh combination ek chhota, common pattern hai, jise kahin bhi duplicates hatane ke liye pehchan lena chahiye, bina sequence kharab kiye.

PART 5 · 4 OF 6

Remembering Where To Save Files

Yeh wahi hissa hai jo Part 1 mein uthaye gaye gap ko band karta hai -- code ke andar ek baar fix kiya hua folder path hatakar, aisi cheez laata hai jo user se poochhi jaati hai, aur chahe to yaad bhi rakhi jaati hai. Yeh ek genuinely chhote idea par tika hai: disk par ek chhoti si file jo runs ke beech bachi rehti hai, jisme ek saved value hoti hai.

Padhta/Likhta hai: notebook ke saath ek chhoti JSON file

Deta hai: ek saved path, ya None agar kuch nahi hai

Use karta hai: json, os

```
import json, os

CONFIG_FILE = "downloader_config.json"
# Yeh chhoti si file save hoti hai notebook ke saath waale folder mein --
# isi mein "pichli baar ka default folder" yaad rakha jaata hai.

def load_saved_folder():
    # Kaam: agar pehle kabhi ek folder "default" set kiya gaya tha, to woh wapas do.
    # Input: kuch nahi
    # Output: folder path (string) agar mila, warna None
    if os.path.exists(CONFIG_FILE):
        with open(CONFIG_FILE, "r") as f:
            data = json.load(f)          # JSON file ko Python dictionary mein badal do
            return data.get("default_folder")
            # .get() safe hai -- key na ho to crash nahi hoga, seedha None milega

    return None # pehli hi baar chal raha hai -- kuch bhi saved nahi hai abhi

def save_folder_as_default(path: str):
    # Kaam: user ne "Yes, isse default bana do" chuna -- to yeh path file mein likh do.
    # Input: path (string) -- jo folder user ne diya
    # Output: kuch return nahi karta -- sirf disk par likhta hai (ek "side-effect")
    with open(CONFIG_FILE, "w") as f:
        json.dump({"default_folder": path}, f)
        # json.dump() dictionary ko file mein JSON text ki tarah likh deta hai
```

In do functions ke around, kuch bhi shuru hone se pehle ek chhota widget group chalta hai: ek text box jo

`load_saved_folder()` kuch return kare to pehle se bhara milega, ek Yes/No choice "isse default banayein?" ke liye, aur ek Confirm button. Yeh group confirm hone ke baad hi Part 4 wala file upload widget use ho paata hai -- sequence hai: pehle folder, phir links, phir download.

SITUATION	STUDENT KO KYA DIKHTA HAI
Sach mein pehli baar -- abhi tak koi config file nahi	Ek khaali text box, seedha ek folder poocha jaata hai, saath mein Yes/No choice
Baad ki koi run, default pehle se saved hai	Text box mein pehle se saved path bhara hua, phir bhi editable aur confirmable

EK CHHOTA PAR ZAROORI BADLAAV: DEST_DIR AB FIXED NAHI REHTA

Ek baar folder program chalte hue chuna ja sakta hai, to woh file ke top par ek baar likha hua constant nahi reh sakta. Woh ek variable ban jaata hai jise ek button ka callback update karta hai -- bilkul waise hi jaise `uploaded_urls` pehle se update hota hai jab koi file attach hoti hai -- state ka ek aur example jo run ke *dauraan* badalta hai, run se *pehle* fix hone ki jagah.

PART 5 · 5 OF 6

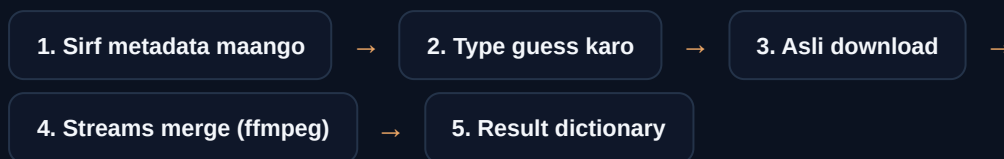
Fetching And Converting: The Core Function

Leta hai: url (string), dest_dir (string)

Deta hai: ek dictionary -- hamesha, kabhi exception nahi

Use karta hai: yt_dlp.YoutubeDL, os

`download_one(url, dest_dir)` poore project ka core hai, aur iska sabse important design decision yeh hai: **yeh kabhi exception raise nahi karta.** Har mumkin failure -- ek private video, ek delete ho chuki post, ek network error -- pakad li jaati hai aur ek dictionary ke andar data ki tarah wapas bheji jaati hai, call stack ke upar kabhi phenki nahi jaati. Yehi hai jo sau links ke batch ko unmein se das ke fail hone par bhi zinda rakhta hai, baaki nabbe kabhi interrupt hue bina.



```

def download_one(url: str, dest_dir: str) -> dict:
    # Kaam: EK link download karo. Kabhi crash mat karo -- sab result dictionary mein bhejo.
    result = {
        "url": url, "platform": detect_platform(url), "type": "Unknown",
        "status": "failed", "title": None, "path": None,
        "size_mb": None, "error": None,
    }
    # Result pehle hi "failed" maan ke bana lo -- jaise kaam hoga, update karte jaayenge

    try:
        # ---- Pehla pass: sirf metadata maango, download nahi ----
        with yt_dlp.YoutubeDL({"quiet": True, "skip_download": True}) as ydl:
            info = ydl.extract_info(url, download=False)
            # skip_download=True -- sirf title/duration, video ka ek bhi byte nahi

            if info is None:
                raise ValueError("No metadata (private/unavailable video?)")

            result["type"] = detect_video_type(url, info)
            title = sanitize_filename(info.get("title", "untitled_video"))

            # ---- Doosra pass: ab asli download ----
            download_opts = {
                "format": FORMAT_SELECTOR,      # Part 2 wala size-based fallback chain
                "outtmpl": os.path.join(dest_dir, f"{title}.{ext}s"),
                "noplaylist": True,             # zaroori! ek link = ek video, poori playlist nahi
                "merge_output_format": "mp4",   # yeh "hamesha MP4" wala promise poora karta hai
            }
            with yt_dlp.YoutubeDL(download_opts) as ydl:
                info_dl = ydl.extract_info(url, download=True)
                final_path = ydl.prepare_filename(info_dl)

                if os.path.exists(final_path):
                    size_mb = os.path.getsize(final_path) / (1024 * 1024)
                    result["status"] = "success"
                    result["path"], result["size_mb"] = final_path, round(size_mb, 2)
                else:
                    raise FileNotFoundError("Download finished but file was not found")

    except Exception as e:
        result["error"] = f"{type(e).__name__}: {e}"
        # Yahan pakad lo -- result "failed" rahega, poora program nahi girega

    return result # HAMESHA ek dictionary -- kabhi bhi Exception nahi phenkta

```

`noplaylist: True` ek accidental पूरी-playlist download ko rokta hai, agar paste kiya hua ek link secretly kisi playlist ka hissa nikle. `merge_output_format: "mp4"` wahi cheez hai jo Part 1 wala "hamesha MP4" promise poora karta hai -- kuch platforms video aur audio ko do alag streams ki tarah bhejte hain, aur yeh setting download engine ko bolti hai unhe ek .mp4 container mein jodo -- iske peeche **ffmpeg** kaam karta hai, isi liye Part 7 mein ffmpeg ek required install ki tarah list kiya gaya hai.

FAILURE KO DATA BANA DO, CRASH NAHI

Yeh pattern -- risky kaam ko try/except mein lapetna, aur dono situations mein ek structured result wapas dena -- naam se yaad rakhne layak hai. Yehi wajah hai ki ek marad hua Instagram link, sau mein se, poore batch job ko kabhi nahi rokta. Caller ko apna khud ka defensive code bhi nahi likhna padta; woh seedha bharosa kar sakta hai ki `download_one` hamesha kuch usable hi wapas dega.

PART 5 · 6 OF 6

The Supporting Cast: Running The Interface

Ek mutthi bhar bache hue functions sirf Part 4 mein describe kiya gaya interface chalane ke liye exist karte hain -- inmein se koi bhi download logic ko seedha touch nahi karta, aur yeh khud ek achhi habit hai notice karne layak: asli kaam (`download_one` aur uske saathi) kabhi ipywidgets par depend nahi karta, isliye wahi logic bina kisi interface ke ek plain script ke andar bhi reuse ho sakta hai, jaisa Part 7 dikhata hai.

FUNCTION	KAB CHALTA HAI	ISKA EK KAAM
<code>_extract_uploaded_bytes</code>	Upload handler ke andar se call hota hai	Uploaded file ke bytes padhta hai, ipywidgets ke do alag versions ko safely handle karte hue
<code>_on_upload_change</code>	Jab koi file upload hoti hai	Text decode karta hai, <code>parse_links()</code> call karta hai, confirmation banner dikhata hai, Start unlock karta hai
<code>render_results_table</code>	Har link khatam hone ke baad	Colour-coded HTML table banata hai -- ek pure function, same input hamesha same output deta hai
<code>save_summary</code>	Poora batch khatam hone ke baad, ek hi baar	Har success aur failure ki timestamped .txt report likhta hai
<code>on_start_clicked</code>	Jab Start button click hota hai	Orchestrator -- har link par loop karta hai, <code>download_one</code> call karta hai, progress bar aur table update karta hai

DO SAFETY NETS, JAAN-BOOJH KAR

`on_start_clicked` apne `download_one` ke call ko apne khud ke ek extra try/except mein lapetta hai, jabki `download_one` already kabhi raise nahi karta. Yeh jaan-boojh kar rakhi gayi redundancy hai, laapraavaahi nahi -- ek doosra net us case ke liye jab kabhi future mein code ka koi edit chain mein kahin galti se crash wapas le aaye. Belt aur suspenders dono lagana, un codes mein ek legitimate engineering choice hai jo logon ke saamne unattended chalne wale hain, over-caution nahi.

PART 6

What This Project Cannot Do

Short aur precise -- room ke saamne chalne se pehle yaad rakhne layak.

Har tool ki apni boundary hoti hai, aur ek achha engineer un boundaries ko chhupata nahi -- unhe pehle se jaan kar rakhta hai. Yeh section short aur precise hai, kyunki yeh un cheezon ki list hai jo class mein live demo karte waqt yaad rehni chahiye.

- **Private, delete ho chuki, ya login-required videos hamesha fail hongii.** Koi bhi tool woh nahi laa sakta jise platform khud, bina authentication ke, dena hi mana kar deta hai -- yeh ek hard boundary hai, bug nahi.
- **Platform mein badlaav download raatoorat tod sakte hain.** Download engine har site ki internal structure samajh kar kaam karta hai; jab koi platform woh structure badal deta hai, downloads tab tak fail hote hain jab tak engine update na ho -- aksar sirf ek `pip install -U yt-dlp` door.
- **ffmpeg install na ho to MP4 ki guarantee nahi.** "Hamesha MP4" wala promise ffmpeg machine par hone par depend karta hai; agar woh nahi hai, to kuch downloads split, un-merged format mein reh sakte hain.
- **Size cap ek best effort hai, guarantee nahi.** Download hone se pehle platform jo size batata hai, woh aksar ek estimate hoti hai, measurement nahi -- result kabhi-kabhi intended ceiling se thoda upar ja sakta hai.
- **Ek .txt file, ek run.** Tool multiple uploads ko aapas mein merge nahi karta, aur na hi kisi folder mein naye files aane ka wait karta hai.
- **Playlists aur poore channels jaan-boojh kar exclude kiye gaye hain.** `noplaylist=True` ka matlab hai: ek link hamesha ek video maana jaata hai, jaan-boojh kar.
- **Interruption ke baad koi resume nahi.** Agar ek batch beech mein rok diya jaaye, to jo files download ho chuki hain woh rehti hain, lekin baaki ke liye poori run dobara shuru karni padegi.
- **Platform Terms of Service ab bhi apply hote hain.** Zyadatar platforms ke terms is tarah download karne ko restrict karte hain. Yeh project personal, offline, fair-use situations ke liye bana hai -- kisi aur ka content redistribute karne ke liye nahi.

KISI BHI LIVE DEMO SE PEHLE KYA CHECK KAREIN

Confirm kar lijiye ki demo machine par ffmpeg actually installed hai, class ki subah `pip install -U yt-dlp` se yt-dlp refresh kar lijiye, aur ek known-working public link fallback ki tarah taiyaar rakhiye -- agar koi chuna hua example link chupke se private ho gaya ho jab se chuna gaya tha.

PART 7

Before Class 2 — Getting Ready

Kya install karna hai, aur roughly kitna time lagega.

Neeche di gayi cheezein Class 2 se pehle ho jaani chahiye, uske dauraan nahi -- installs aur system tools mein kuch minute lag sakte hain har ek mein, aur unhe raat pehle sambhalna, classroom ke saamne live karne se kahin behtar hai.

TOOL	INSTALL COMMAND	NOTES
Python 3.9+	python.org se, ya Anaconda se	Agar pehle se installed hai to skip kariye
Jupyter Notebook / Lab	<code>pip install notebook</code>	Ya browser mein Google Colab use kariye -- locally kuch bhi install karne ki zaroorat nahi
yt-dlp	<code>pip install -U yt-dlp</code>	Core engine; -U isse current rakhta hai, har session se pehle dobara chalane layak
ipywidgets	<code>pip install ipywidgets</code>	Upload box, buttons, aur progress bar ke liye zaroori
ffmpeg	Windows: <code>winget install ffmpeg</code> · Mac: <code>brew install ffmpeg</code> · Linux: <code>sudo apt install ffmpeg</code>	Yeh pip package NAHI hai -- ek alag system tool, streams ko .mp4 mein merge karne ke liye zaroori

Local notebook chalane ke liye, project folder mein ek terminal kholiye aur `jupyter notebook` chalayiye -- yeh browser tab mein interface khol dega. Google Colab ko sign in karne ke alawa kuch bhi locally install nahi karna padta, halaanki `yt-dlp` ko phir bhi Colab session ke andar hi wahi `pip install` command se install karna padta hai -- sirf ffmpeg wahan pehle se installed milta hai.

Upar di gayi saari cheezon ka ek fresh install normal connection par lagbhag paanch se das minute leta hai. Ise ek baar, class se pehle, aaram se kar lena, demo ke beech mein slow download se class ka time barbaad hone se bacha leta hai.

PART 7 · CONTINUED

The Same Project, As A Plain Script

Ek notebook explore karne ke liye behtareen hai, lekin ek finished tool aksar ek plain `.py` script ki tarah bhi ship kiya jaata hai -- kuch aisa jo koi terminal se chala sake, bina kisi notebook ke. Is project ko convert karna sunne mein jitna bada lagta hai, utna hai nahi, kyunki download karne wali logic pehle se hi ipywidgets par depend nahi karti thi.

NOTEBOOK MEIN	EK PLAIN SCRIPT MEIN
Ek FileUpload widget	<code>input("Path to your links file: ")</code> , ya argparse se ek command-line argument
Ek live, colour-coded HTML table	Har result ke liye ek <code>print()</code> line, saath mein wahi saved .txt summary
Ek IntProgress widget	Ek simple counter, <code>f"{idx}/{total}"</code> ki tarah print kiya hua
Cells execute hote hi automatically chalta hai	<code>if __name__ == "__main__":</code> ke andar lapetta hua, taaki sirf tabhi chale jab file directly run ho

```
# script_version.py -- ekdum wahi logic, bina ipywidgets ke

def main():
    dest_dir = input("Kis folder mein save karna hai? ")
    txt_path = input("Links wali .txt file ka path: ")

    with open(txt_path, "r", encoding="utf-8") as f:
        urls = parse_links(f.read())    # wahi function jo notebook mein tha

    print(f"{len(urls)} link(s) mile. Shuru kar rahe hain...")
    results = []
    for idx, url in enumerate(urls, start=1):
        print(f"[{idx}/{len(urls)}] {url[:60]}")
        results.append(download_one(url, dest_dir))    # yeh bhi bilkul wahi function hai

    save_summary(results, dest_dir)
    print("Ho gaya. Summary file save ho chuki hai.")

if __name__ == "__main__":
    main()
    # Yeh line isliye zaroori hai: agar koi is file ko "import" kare kisi doosri
    # file mein, to main() apne aap nahi chalega -- sirf seedha run karne par chalega.
```

Baaki sab kuch -- `detect_platform`, `sanitize_filename`, `parse_links`, `download_one` -- bina kisi badlaav ke carry over hota hai. Yeh ki woh pehle se hi interface par depend nahi karte the, khud ek chhota, kaam ka lesson hai: logic aur interface ko alag rakhna, matlab ek ko badle bina doosre ko swap kiya ja sakta hai.

CLOSE

Where You Stand Now

Is page ko abhi ek baar padhiye, aur ek baar phir, Class 2 shuru hone se thoda pehle.

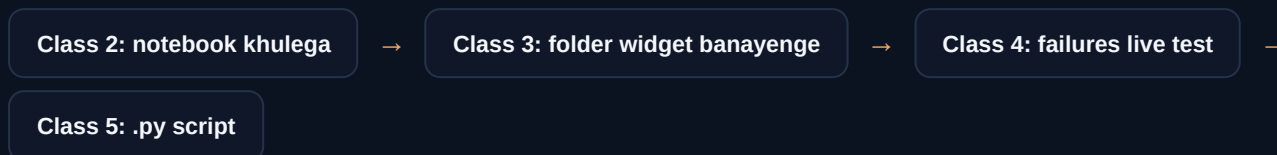
Yeh session yahin khatam hota hai, notebook ek baar bhi khole bina. Jo bhi cover kiya gaya -- ecosystem, regex patterns, event flow, aur har function ka exact behaviour -- sab isi ek moment ki taraf badh raha tha: agli class mein asli code ke saamne baithna, aur har piece ko pehchan lena.

SELF-CHECK

Class 2 ke liye taiyaar hain agar inka jawab yaad se, plain shabdon mein, bina code ke diya ja sake:

- Regex pattern ke andar `https?` ka matlab kya hai, aur exactly question mark kya karta hai?
- `download_one` kabhi exception kyun nahi raise karta -- iski jagah har baar kya return karta hai?
- `download_one` ke andar probe pass aur download pass mein kya farak hai?
- `noplaylist=True` kyun matter karta hai, aur bina iske kya galat ho sakta hai?
- `merge_output_format="mp4"` peeche kis cheez par bharosa karta hai, actually kaam karne ke liye?
- `parse_links` ek set aur ek list saath kyun use karta hai, sirf ek ki jagah?

AAGE KA RAASTA



CLASS	KYA HOGA
Class 2	Asli notebook pehli baar khulega -- is guide ke saath side by side har cell chalayenge
Class 3	Destination-folder widget aur saved-default file, hands-on banayenge
Class 4	Failure cases live test karenge -- ek private video, ek broken link, ek missing ffmpeg install
Class 5	Project ko ek standalone .py script ki tarah package karke terminal se chalayenge

WOH EK LINE JO IS POORI CLASS KO BAANDH KE RAKHTI HAI

Ek batch job utna hi bharosemand hota hai jitni achhi uski worst-case handling hoti hai -- ek galat link, ek missing tool, ek galat type kiya hua folder. Is project mein sab kuch isi liye bana hai taaki ek akeli failure hamesha report ki ek line bani rahe, kabhi ek ruki hui run nahi.